DATA STRUCTURES
HOMEWORK EX_02
ROLL_N0:2025115046

1.#include <stdio.h>

```
int main()
{
    int a[5];
    int *ptr;
    int i;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++)
    {
        scanf("%d", &a[i]);
    }

    ptr = &a[0];

    printf("Array elements using pointer:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", *ptr);
        ptr++;
    }

    return 0;
}
```

Output:

```
Enter 5 elements:
10
20
30
40
50

Array elements using pointer:
10 20 30 40 50
```

2.Modify array elements using pointer arithmetic

```
#include <stdio.h>

int main()
{
    int a[5];
    int *ptr;
    int i;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++)
    {
        scanf("%d", &a[i]);
    }

    ptr = a;

    printf("Original array:\n");
```

```
    for(i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }

    /* Modify each element */
    for(i = 0; i < 5; i++)
    {
        *(ptr + i) = *(ptr + i) + 10;
    }

    printf("\nModified array:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }

    return 0;
}

Output:
Enter 5 elements:
10
20
30
40
50

Original array:
10 20 30 40 50

Modified array:
20 30 40 50 60

3.Function to calculate sum and product

#include <stdio.h>

void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}

int main()
{
    int num1, num2;
    int sum, product;

    printf("Enter two numbers: ");
    scanf("%d %d", &num1, &num2);

    calculate(num1, num2, &sum, &product);

    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);

    return 0;
}

Output:
Enter two numbers: 12 5
Sum = 17
```

```
Product = 60
```

4.Array of structures + Dynamic Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>

struct Student
{
    int id;
    char name[30];
    float mark;
};

int main()
{
    struct Student *ptr;
    int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);

    ptr = (struct Student *)malloc(n * sizeof(struct Student));

    if(ptr == NULL)
    {
        printf("Memory allocation failed");
        return 1;
    }

    printf("\nEnter student details:\n");

    for(i = 0; i < n; i++)
    {
        printf("\nStudent %d\n", i + 1);

        printf("Enter ID: ");
        scanf("%d", &ptr[i].id);

        printf("Enter Name: ");
        scanf("%s", ptr[i].name);

        printf("Enter Mark: ");
        scanf("%f", &ptr[i].mark);
    }

    /* Modifying member */
    ptr[0].mark = ptr[0].mark + 5;

    printf("\nStudent Details:\n");

    for(i = 0; i < n; i++)
    {
        printf("\nID   : %d", ptr[i].id);
        printf("\nName : %s", ptr[i].name);
        printf("\nMark : %.2f\n", ptr[i].mark);
    }

    free(ptr);

    return 0;
}
```

Output:

```
Enter number of students: 2
```

```
Enter student details:

Student 1
Enter ID: 101
Enter Name: Aruna
Enter Mark: 80

Student 2
Enter ID: 102
Enter Name: Anu
Enter Mark: 90

Student Details:

ID   : 101
Name : Aruna
Mark : 85.00

ID   : 102
Name : Anu
Mark : 90.00
```

5.Function pointer to call two functions

```
#include <stdio.h>

void add(int a, int b)
{
    printf("Addition = %d\n", a + b);
}

void subtract(int a, int b)
{
    printf("Subtraction = %d\n", a - b);
}

int main()
{
    void (*fp)(int, int);

    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    /* Function pointer points to add */
    fp = add;
    fp(a, b);

    /* Function pointer points to subtract */
    fp = subtract;
    fp(a, b);

    return 0;
}
```

```
Output:
Enter two numbers: 20 8
Addition = 28
Subtraction = 12
```

6. Integer pointer + Double pointer

```
#include <stdio.h>
```

```
int main()
{
    int x = 25;

    int *ptr;
    int **dptr;

    ptr = &x;
    dptr = &ptr;

    printf("Value of x = %d\n", x);

    printf("Value using pointer = %d\n", *ptr);

    printf("Value using double pointer = %d\n", **dptr);

    printf("\nAddress of x = %p\n", (void *)&x);

    printf("Address stored in ptr = %p\n", (void *)ptr);

    printf("Address of ptr = %p\n", (void *)&ptr);

    printf("Address stored in dptr = %p\n", (void *)dptr);

    return 0;
}
Output:
Value of x = 25
Value using pointer = 25
Value using double pointer = 25

Address of x = 0x7ffd1234
Address stored in ptr = 0x7ffd1234
Address of ptr = 0x7ffd1230
Address stored in dptr = 0x7ffd1230

7.Floating point array using Selection Sort
#include <stdio.h>

int main()
{
    float a[5];
    float temp;
    int i, j, min;

    printf("Enter 5 floating point numbers:\n");

    for(i = 0; i < 5; i++)
    {
        scanf("%f", &a[i]);
    }

    /* Selection Sort */
    for(i = 0; i < 4; i++)
    {
        min = i;

        for(j = i + 1; j < 5; j++)
        {
            if(a[j] < a[min])
            {
                min = j;
            }
        }
```

```
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    printf("\nSorted array:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%.2f ", a[i]);
    }

    return 0;
}
```

Output:

```
Enter 5 floating point numbers:
5.5
2.2
8.8
1.1
4.4

Sorted array:
1.10 2.20 4.40 5.50 8.80
```

8.Structure Student â ¨ sort based on different fields

```
#include <stdio.h>

struct Student
{
    int id;
    char name[30];
    float grade;
};

int main()
{
    struct Student s[5], temp;
    int i, j, n;

    printf("Enter number of students: ");
    scanf("%d", &n);

    printf("\nEnter student details:\n");

    for(i = 0; i < n; i++)
    {
        printf("\nStudent %d\n", i + 1);

        printf("Enter ID: ");
        scanf("%d", &s[i].id);

        printf("Enter Name: ");
        scanf("%s", s[i].name);

        printf("Enter Grade: ");
        scanf("%f", &s[i].grade);
    }

    /* Sort based on grade - descending */
    for(i = 0; i < n - 1; i++)
    {
```

```
        for(j = i + 1; j < n; j++)
        {
            if(s[i].grade < s[j].grade)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("\nStudents sorted based on grade:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d %s %.2f\n",
               s[i].id,
               s[i].name,
               s[i].grade);
    }

    return 0;
}

Output:
Enter number of students: 3

Student 1
Enter ID: 101
Enter Name: Arun
Enter Grade: 85

Student 2
Enter ID: 102
Enter Name: Anu
Enter Grade: 95

Student 3
Enter ID: 103
Enter Name: Ravi
Enter Grade: 75

Students sorted based on grade:
102 Anu 95.00
101 Arun 85.00
103 Ravi 75.00

9.Sort negative numbers using Radix Sort

#include <stdio.h>

int getMax(int a[], int n)
{
    int max = a[0];
    int i;

    for(i = 1; i < n; i++)
    {
        if(a[i] > max)
            max = a[i];
    }

    return max;
}
```

```
void countingSort(int a[], int n, int exp)
{
    int output[20];
    int count[10] = {0};
    int i;

    for(i = 0; i < n; i++)
        count[(a[i] / exp) % 10]++;

    for(i = 1; i < 10; i++)
        count[i] = count[i] + count[i - 1];

    for(i = n - 1; i >= 0; i--)
    {
        output[count[(a[i] / exp) % 10] - 1] = a[i];
        count[(a[i] / exp) % 10]--;
    }

    for(i = 0; i < n; i++)
        a[i] = output[i];
}

void radixSort(int a[], int n)
{
    int max;
    int exp;

    max = getMax(a, n);

    for(exp = 1; max / exp > 0; exp = exp * 10)
    {
        countingSort(a, n, exp);
    }
}

int main()
{
    int a[5] = {-45, -12, -89, -23, -5};
    int b[5];
    int i;

    /* Convert negative values to positive */
    for(i = 0; i < 5; i++)
    {
        b[i] = -a[i];
    }

    radixSort(b, 5);

    printf("Sorted negative numbers:\n");

    /* Reverse order and make them negative */
    for(i = 4; i >= 0; i--)
    {
        printf("%d ", -b[i]);
    }

    return 0;
}

Output:
Sorted negative numbers:
-89 -45 -23 -12 -5
```

10.Copy one array to another using pointers

```
#include <stdio.h>

int main()
{
    int a[5], b[5];
    int *p, *q;
    int i;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++)
    {
        scanf("%d", &a[i]);
    }

    p = a;
    q = b;

    /* Copy using pointers */
    for(i = 0; i < 5; i++)
    {
        *q = *p;

        p++;
        q++;
    }

    printf("\nOriginal array:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\nCopied array:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", b[i]);
    }

    return 0;
}
```

Output:

```
Enter 5 elements:
10
20
30
40
50

Original array:
10 20 30 40 50

Copied array:
10 20 30 40 50
```